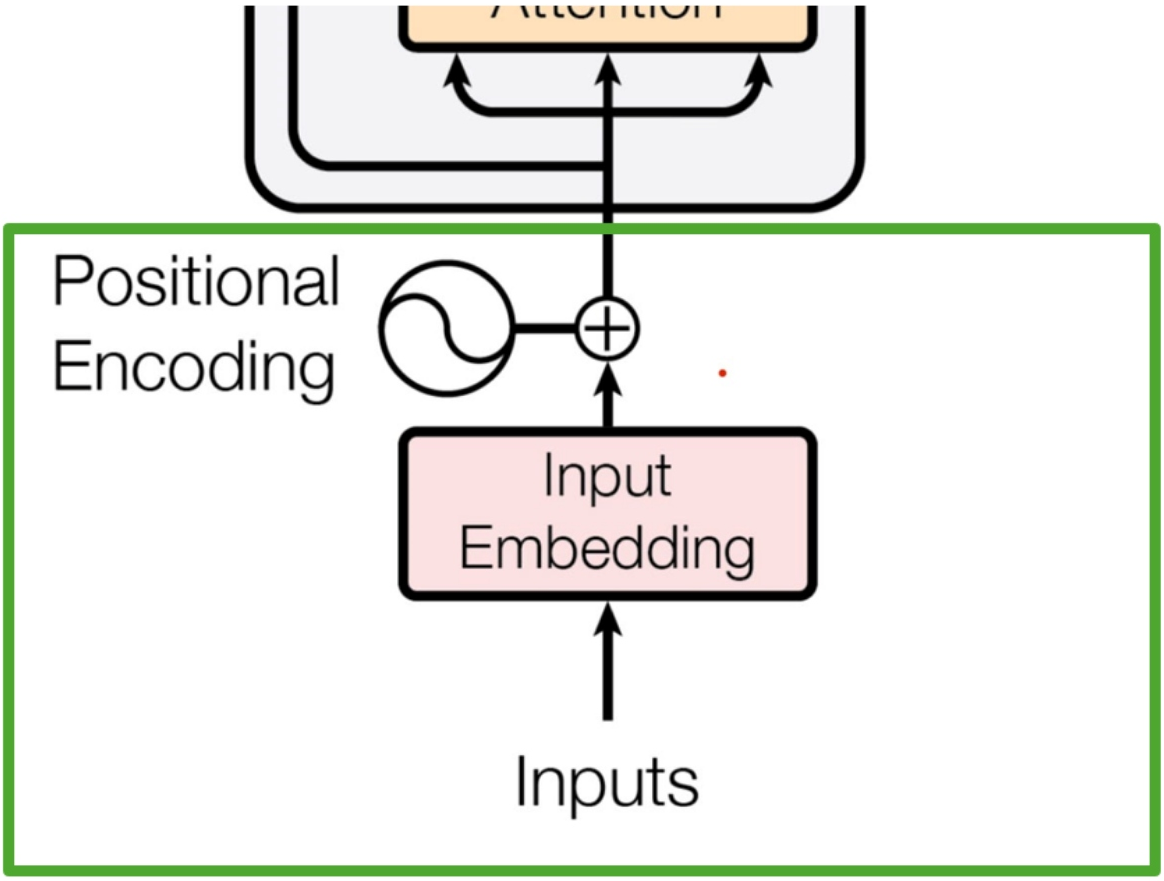
Here's a **clear, end-to-end view of GPT / LLM architecture flow**, broken into **training-time flow** and **inference-time flow** (what actually happens when you type a prompt).

🧠 1. High-Level GPT Architecture


Positional
Encoding
Input
Embedding
Inputs

At its core, GPT is a **decoder-only Transformer** based on the original paper 👉 Transformer Architecture

Core components:

- Tokenizer
- Embedding layer
- Positional encoding
- Stacked Transformer blocks
- Linear + Softmax output

🔄 2. End-to-End Flow (Simple View)

```
Input Text → Tokenization → Embeddings → Transformer Layers → Logits → Softmax → Next Toke
```

🧩 3. Step-by-Step Architecture Flow

(1) Tokenization

- Input text → split into tokens
- Example:

```
"ChatGPT is powerful"
→ ["Chat", "G", "PT", " is", " powerful"]
```

- Uses:
  - BPE (Byte Pair Encoding)
  - SentencePiece

---

(2) Token Embedding + Positional Encoding

Each token becomes a vector:

```
token → embedding vector (e.g., 768 / 4096 dims)
```

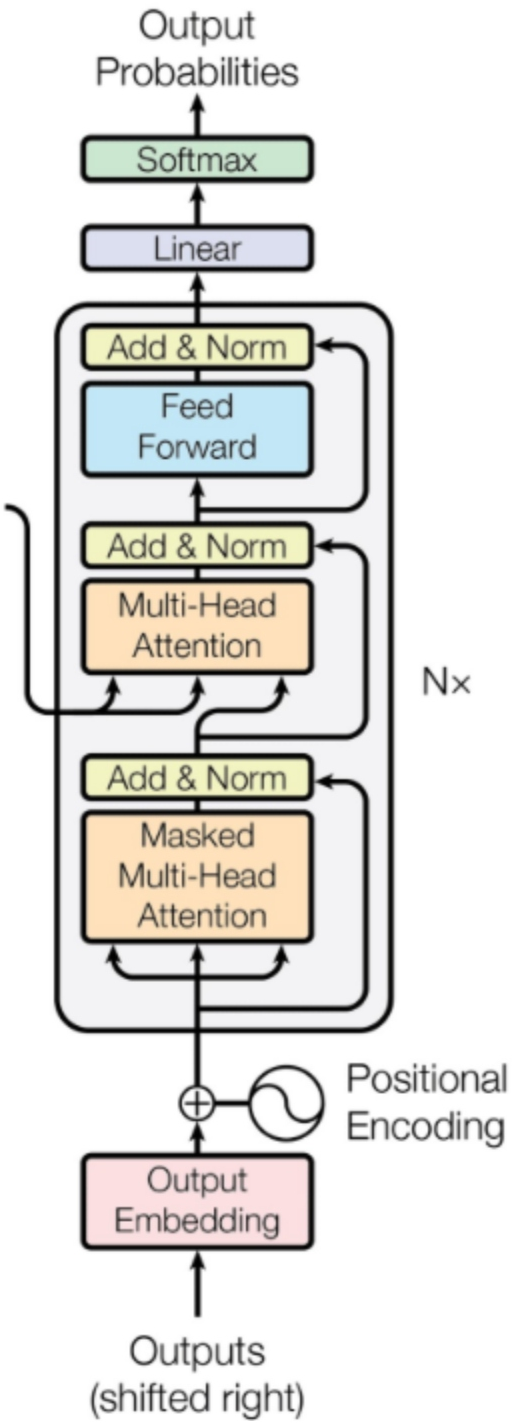
Add position info:

```
Final input = Token Embedding + Positional Encoding
```

👉 Without position, model doesn't understand order.

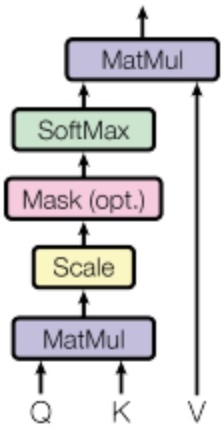
---

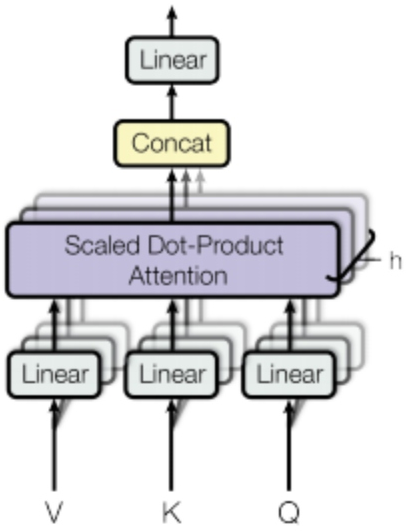
(3) Transformer Blocks (Core Engine)

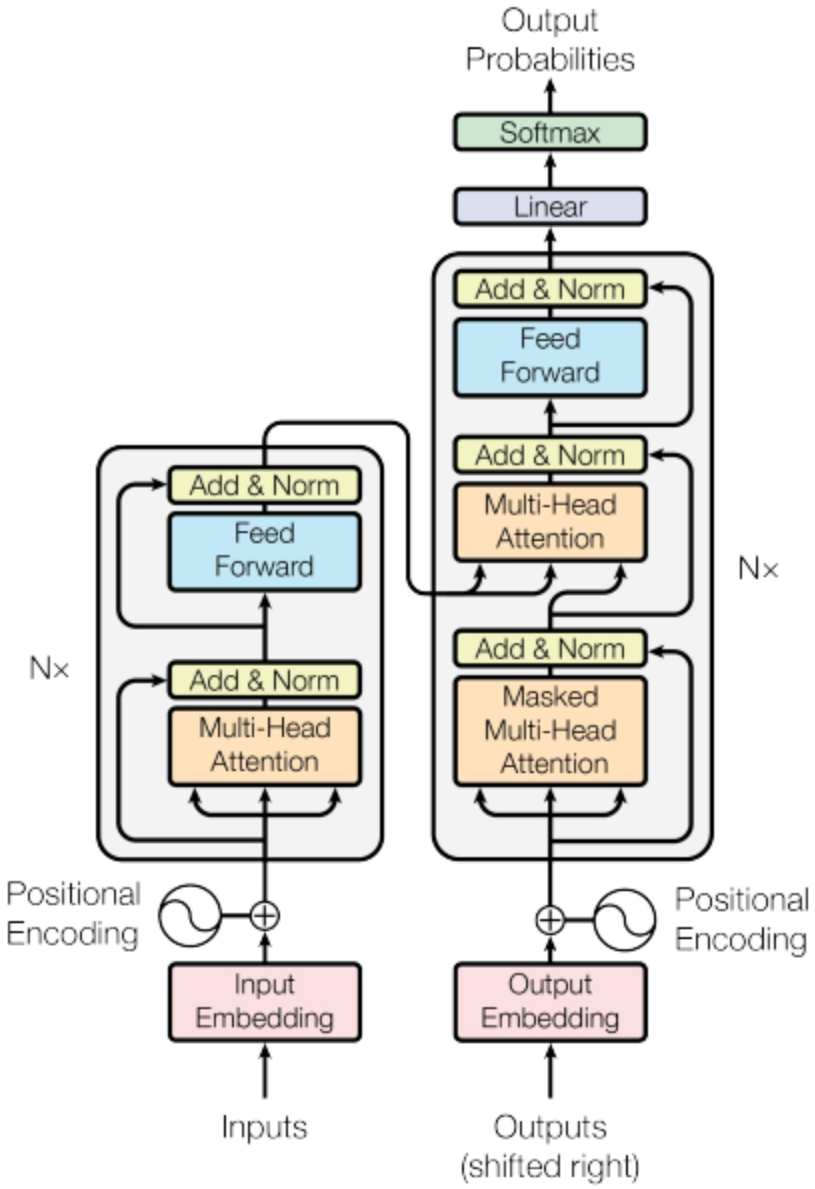


Scaled Dot-Product Attention

Multi-Head Attention


MatMul
SoftMax
Mask (opt.)
Scale
MatMul
Q
K
V


Linear
Concat
Scaled Dot-Product Attention
h
Linear
Linear
Linear
V
K
Q


Output Probabilities
Softmax
Linear
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
Add & Norm
Masked Multi-Head Attention
N×
Add & Norm
Feed Forward
Add & Norm
Multi-Head Attention
N×
Positional Encoding
Positional Encoding
Input Embedding
Output Embedding
Inputs
Outputs (shifted right)

Each block contains:

🔷 (a) Masked Self-Attention

- Every token looks at **previous tokens only**
- Uses:
  - Query (Q)
  - Key (K)
  - Value (V)

Attention formula:

[Attention(Q,K,V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V]

👉 This is where **context understanding happens**

🔷 (b) Multi-Head Attention

- Multiple attention heads in parallel
- Each learns different relationships:
  - grammar
  - meaning
  - dependencies

🔷 (c) Feed Forward Network (FFN)

• Simple MLP applied per token:

```
FFN(x) = max(0, xW1 + b1)W2 + b2
```

🔷 (d) Residual + LayerNorm

• Stabilizes training

• Enables deep networks (100+ layers)

- Enables deep networks (100+ layers)

---

(4) Stacking Layers

- GPT = stack of Transformer blocks

Examples:

- GPT-2: 12–48 layers
- GPT-4+: 100+ layers (approx, not public)

---

(5) Output Layer (Logits)

Final vector → linear layer → vocabulary scores

```
Hidden State → Linear → Logits (size = vocab)
```

---

(6) Softmax → Next Token Prediction

Convert logits → probabilities:

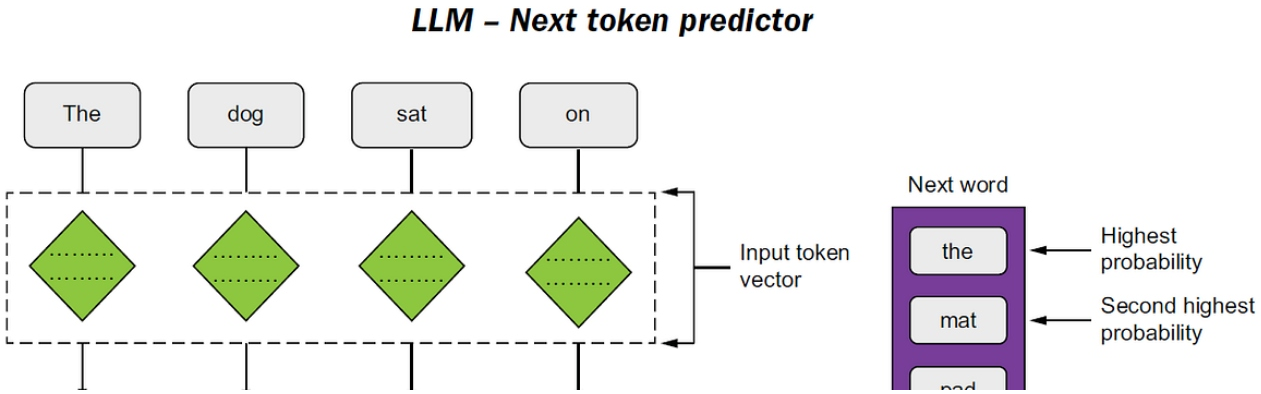
[P(token) = softmax(logits)]

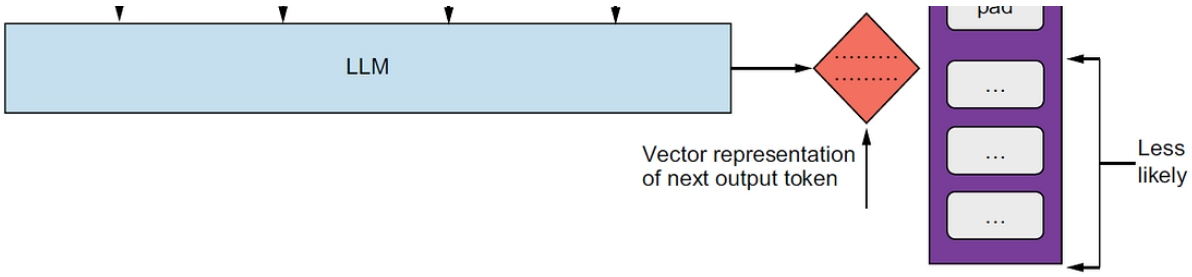
Then:

- pick highest (greedy)
- or sample (temperature, top-k, top-p)

---

4. Training Flow (VERY IMPORTANT)

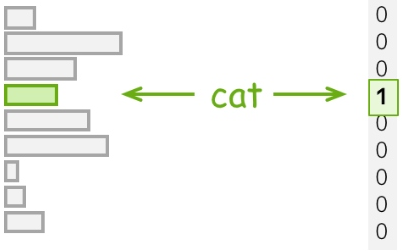




we want the model
to predict this

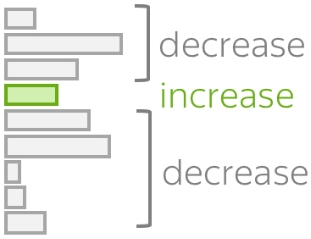
Training example: I saw a cat on a mat <eos>

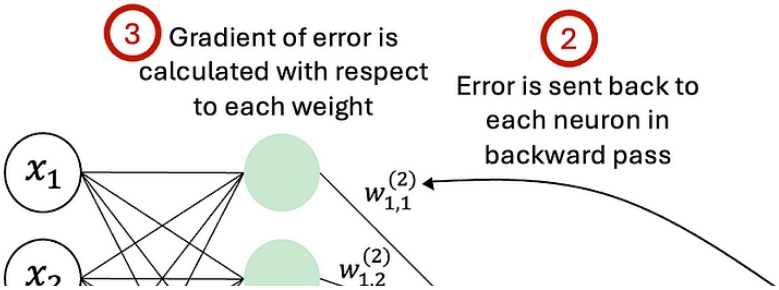
Model prediction: p(∗ | I saw a)

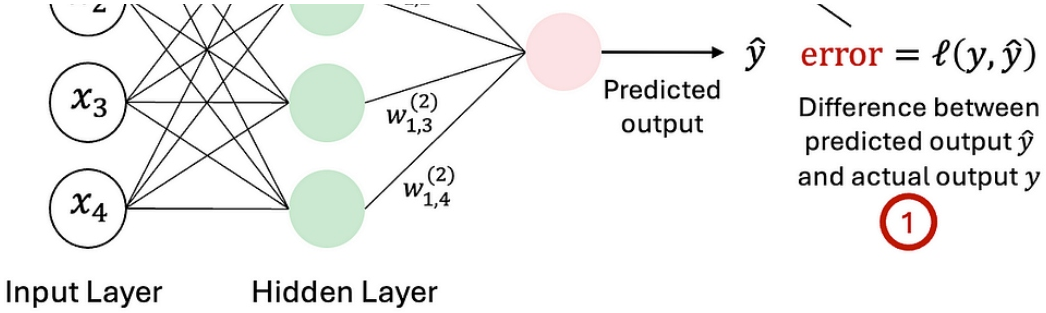


Target

Loss = -log (p(cat)) → min







Objective:

Predict next token

```
Input:  "The cat sat on"
Target: "the"
```

Training Steps:

1. Forward Pass

- Compute predictions

2. Loss Calculation

- Uses: 👉 Cross Entropy Loss

---

3. Backpropagation

- Compute gradients

---

4. Weight Update

- Using:
  - Adam optimizer
  - SGD variants

🔑 Key Insight:

👉 **Weights update ONLY during training, NOT during inference**

⚡ 5. Inference Flow (What Happens in ChatGPT)

When you type:

```
"Explain AI"
```

Step-by-step:

1. Tokenize input
2. Pass through model
3. Predict next token
4. Append token
5. Repeat (autoregressive loop)

```
Input → Predict → Append → Predict → Append → ...
```

🧠 6. Key Concepts That Drive GPT

🔷 Autoregressive Modeling

- Predict next token sequentially

🔷 Self-Attention

- Understand relationships:
  - long-range dependencies
  - context

🔷 Scaling Laws

- More:
  - data
  - parameters

parameters
- compute → better performance

🏗️ 7. Full Architecture Summary (Mental Model)

Think of GPT as:

```
Tokenizer → Vectorizer → Context Engine → Probability Machine → Text Generator
```

Or more technically:

```
Text
 → Tokens
 → Embeddings
 → [Attention + FFN] × N layers
 → Logits
 → Softmax
 → Next token
```

🚀 8. Mapping to Your Background (Important for You)

Since you're a **software architect**, map it like:

System Concept	LLM Equivalent
Input API	Prompt
Encoding layer	Tokenizer
Processing engine	Transformer stack
Decision logic	Attention
Output service	Decoder (softmax sampling)
Training pipeline	CI/CD + optimization loop

🔥 If you want next level

🔥 If you want next level

I can go deeper into:

- Q/K/V **numerical example**
- Full **matrix math walkthrough**
- How **RAG plugs into this architecture**
- How to **build mini GPT from scratch**
- Infra view (GPU, distributed training)

Just tell me 👍